

Token retention using http-only cookie

Task Context

Need to implement sub-domain redirection in case of organisational sign-in.

Expected usage:

- A common login page.
 - Following redirection after login for:
 - Super admin → Root domain
 - College admin → Sub-domain (organisation-based)
-

Initial Understanding

Assumptions:

- After successful login, simply redirect to the sub-domain.
- Existing logic will handle sessions.
- Zustand would work across the root domain and sub-domains.

Realisations:

- Sub-domains have different independent origins.
 - State management libraries are limited to one origin only.
-

Knowledge gap identified

- How to forward tokens on successful login to the desired sub-domain.
-

Exploration and research

- Studied about http-only cookies.
- A common pattern was to send tokens as cookies on successful login and later fetch session details using a separate API.

Mental model built

- Cookie saving layer: The root domain sign-in page where tokens extracted from a successful sign-in response are sent/ saved as cookies.
 - Cookie access and session fetch layer: The sub-domain dashboard page where session details are fetched using a dedicated API.
 - API layer for sending session details: The backend API component that fetches tokens from cookies and sends db retrieved session details to the frontend.
-

Implementation journey

Step 1: Added redirection logic after successful login to desired sub-domain.

```
● window.location.href = redirectPath;
```

Step 2: Inside fetch wrapper apiClient.ts, added cookie transmission across sub-domains using credentials: 'include'.

Step 3: Inside login auth controller in backend, removed tokens from response json, instead set them as response cookie and sent slug for the sub-domain.

```
res.cookie("access_token", accessToken, {  
  httpOnly: true,  
  secure: false, // true in production HTTPS  
  sameSite: "lax",  
  domain: ".lvh.me",  
  path: "/",  
  maxAge: ACCESS_TOKEN_EXPIRES_IN_SECONDS * 1000,  
});
```

Step 4: An API added in auth controllers to fetch session details, based on token decryption.

Step 5: API triggered on AuthGuard page to fetch session details.

Step 6: Changed logout implementation, which now clears the desired cookies when triggered.

Step 7: Ensured slug persistence throughout journey by passing slug fetched from email.

Root domain → Backend (inside req body) → Sub-domain frontend (inside Login response)

Problems faced

- Middlewares mismatched after migrating to cookie auth.
 - Slug fetching for sub-domain from url became obsolete as now we are handling login on root domain, which earlier we did on subdomain for organisational login.
-

Final architecture

- Login attempted
 - Tokens set as cookies by backend
 - Root redirects to sub-domain on login
 - Sub-domain request for session details using cookies
 - Backend decodes tokens and sends details back
-

Key learnings

- For production-level scalable and multi subdomain SaaS products, always prefer the http-only cookie method for token retention.
 - It offers better security, compared to state management libraries as cookies are inaccessible to Js.
-

Future improvements

- Assess all APIs and check for the ones relying on old token architecture and adjust them according to the new cookie-based architecture.